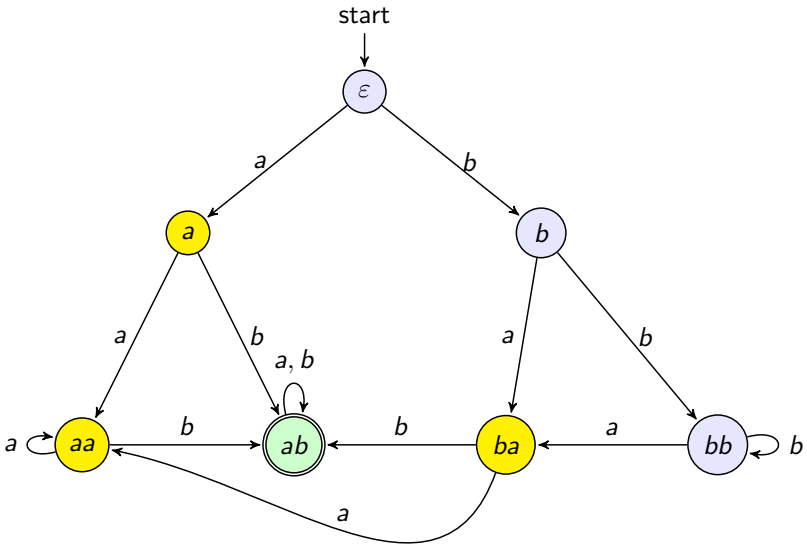
CS 310 : Automata Theory

S. Krishna

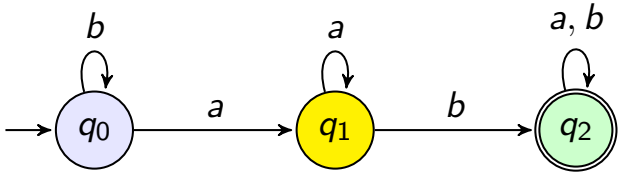
From Jaffe to Minimization

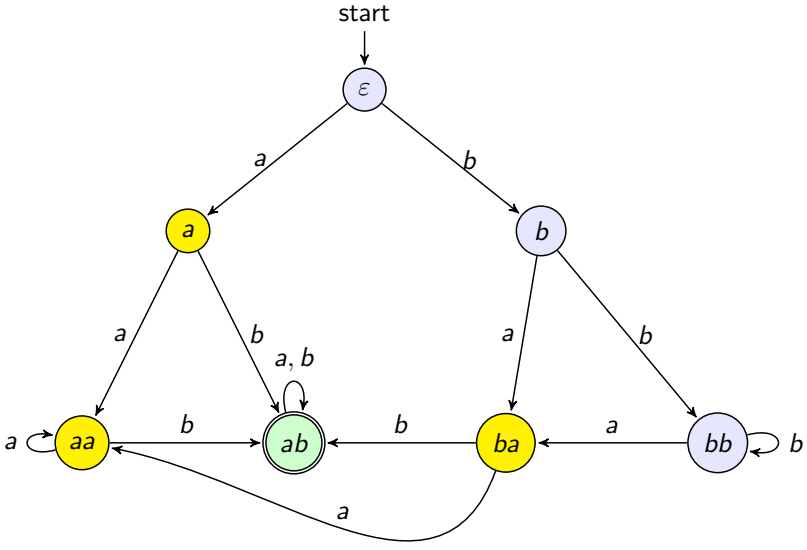
- Jaffe's construction gives *an* automaton, not necessarily the smallest one.
- Our 7-state automaton is collapsible.
  - for any two blue states q_1, q_2, $\forall z \in \Sigma^*$, $\widehat{\delta}(q_1, z) \in L \Leftrightarrow \widehat{\delta}(q_2, z) \in L$
  - for any two yellow states q_1, q_2, $\forall z \in \Sigma^*$, $\widehat{\delta}(q_1, z) \in L \Leftrightarrow \widehat{\delta}(q_2, z) \in L$



From Jaffe to Minimization

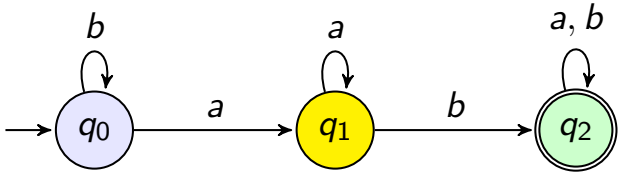
- Jaffe's construction gives *an* automaton, not necessarily the smallest one.
- Our 7-state automaton is collapsible.
  - for any two blue states q_1, q_2, $\forall z \in \Sigma^*$, $\widehat{\delta}(q_1, z) \in L \Leftrightarrow \widehat{\delta}(q_2, z) \in L$
  - for any two yellow states q_1, q_2, $\forall z \in \Sigma^*$, $\widehat{\delta}(q_1, z) \in L \Leftrightarrow \widehat{\delta}(q_2, z) \in L$

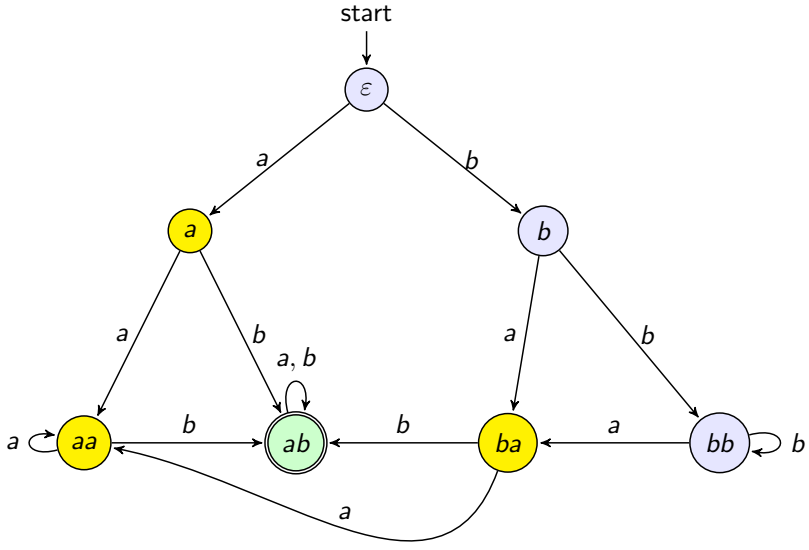




From Jaffe to Minimization

- Jaffe's construction gives *an* automaton, not necessarily the smallest one.
- Our 7-state automaton is collapsible.
  - for any two blue states q_1, q_2, $\forall z \in \Sigma^*$, $\widehat{\delta}(q_1, z) \in L \Leftrightarrow \widehat{\delta}(q_2, z) \in L$
  - for any two yellow states q_1, q_2, $\forall z \in \Sigma^*$, $\widehat{\delta}(q_1, z) \in L \Leftrightarrow \widehat{\delta}(q_2, z) \in L$





- can we build the **unique smallest DFA** for any regular language?

DFA Equivalence and Minimization

- Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \to Q$)
- $L(A) = L(A, q_0)$

DFA Equivalence and Minimization

- Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \to Q$)
- $L(A) = L(A, q_0)$
- Two states q_1, q_2 in A are equivalent if $L(A, q_1) = L(A, q_2)$.

DFA Equivalence and Minimization

- Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \to Q$)
- $L(A) = L(A, q_0)$
- Two states q_1, q_2 in A are equivalent if $L(A, q_1) = L(A, q_2)$.
- A state q_1 in DFA A_1 is equivalent to state q_2 in DFA A_2 if $L(A_1, q_1) = L(A_2, q_2)$.
- Two DFAs A_1, A_2 are equivalent if $L(A_1) = L(A_2)$

DFA Equivalence and Minimization

- Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \to Q$)
- $L(A) = L(A, q_0)$
- Two states q_1, q_2 in A are equivalent if $L(A, q_1) = L(A, q_2)$.
- A state q_1 in DFA A_1 is equivalent to state q_2 in DFA A_2 if $L(A_1, q_1) = L(A_2, q_2)$.
- Two DFAs A_1, A_2 are equivalent if $L(A_1) = L(A_2)$

DFA Equivalence

For every DFA, there exists a unique (upto state naming) minimal DFA.

Minimizing DFAs

Two observations:

- Unreachable states can be removed. This does not change the language accepted.
- Merging equivalent states. This also does not change the language accepted.

Minimizing DFAs

Two observations:

- Unreachable states can be removed. This does not change the language accepted.
- Merging equivalent states. This also does not change the language accepted.

Algorithms:

1. BFS or DFS to identify reachable states and pruning out the rest
2. Table-filling algorithm by E.F.Moore

Table-filling Algorithm

- Two states are distinguishable if they are not equivalent
- Formally, states q_1, q_2 are distinguishable if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w)$, $\hat{\delta}(q_2, w)$ is an accepting state.

Table-filling Algorithm

- Two states are distinguishable if they are not equivalent
- Formally, states q_1, q_2 are distinguishable if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w)$, $\hat{\delta}(q_2, w)$ is an accepting state.
- Table-filling is recursive discovery of distinguishable pairs of states.

Table-filling Algorithm

- Two states are distinguishable if they are not equivalent
- Formally, states q_1, q_2 are distinguishable if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w)$, $\hat{\delta}(q_2, w)$ is an accepting state.
- Table-filling is recursive discovery of distinguishable pairs of states.
- Base case : (p, q) is distinguishable if $p \in F$ and $q \notin F$ or $p \notin F, q \in F$. (why?)

Table-filling Algorithm

- Two states are distinguishable if they are not equivalent
- Formally, states q_1, q_2 are distinguishable if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w)$, $\hat{\delta}(q_2, w)$ is an accepting state.
- Table-filling is recursive discovery of distinguishable pairs of states.
- Base case : (p, q) is distinguishable if $p \in F$ and $q \notin F$ or $p \notin F, q \in F$. (why?)
- Inductive hypothesis : (p, q) is distinguishable if $\delta(p, a)$ and $\delta(q, a)$ are distinguishable for some $a \in \Sigma$. (why?)

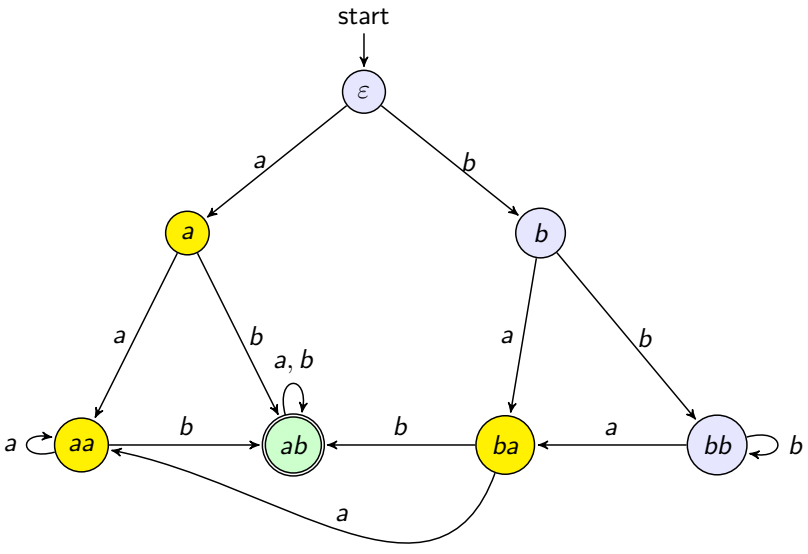
Table-filling Algorithm

1. Distinguishable $= \{(p, q) \mid \text{exactly one of } p, q \in F\}$
2. Repeat until no new pair is added:
   - for every $a \in \Sigma$ and every pair (p, q):
     add (p, q) to Distinguishable if $(\delta(p, a), \delta(q, a)) \in$ Distinguishable.
3. Return Distinguishable.

Let us try an example.

Our 7-state machine

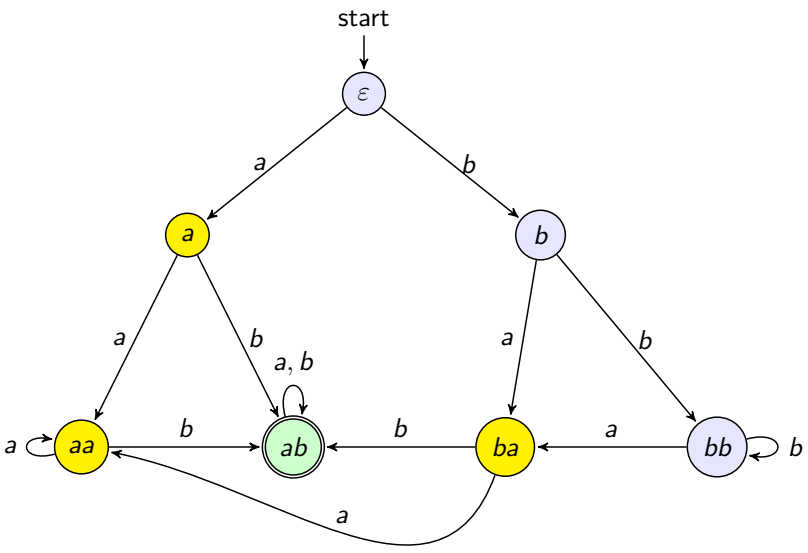
Recall states $\varepsilon, a, b, aa, ab, ba, bb$, accepting $= \{ab\}$. Gray cells are not real pairs (same state, or already counted in another row).



	ε	a	b	aa	ab	ba
a						
b						
aa						
ab						
ba						
bb						

Base case

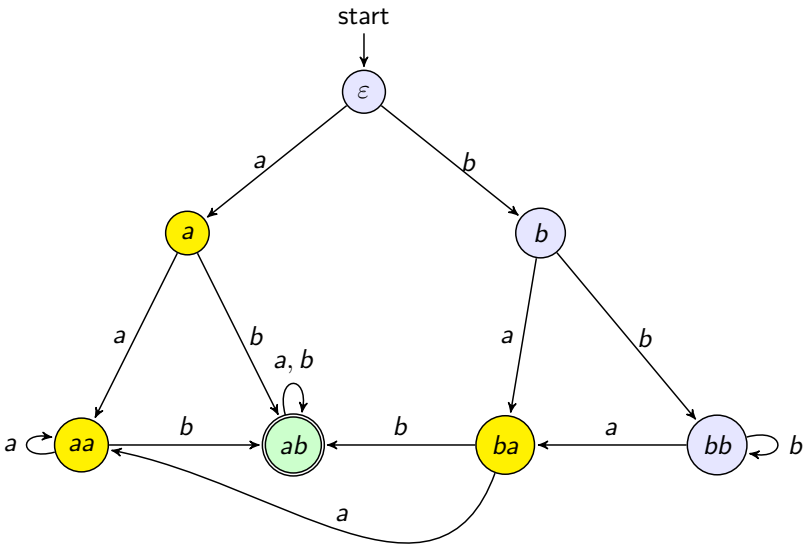
ab is the only accepting state, so every pair involving it is immediately distinguishable.



	ε	a	b	aa	ab	ba
a						
b						
aa						
ab						
ba						
bb						

Base case

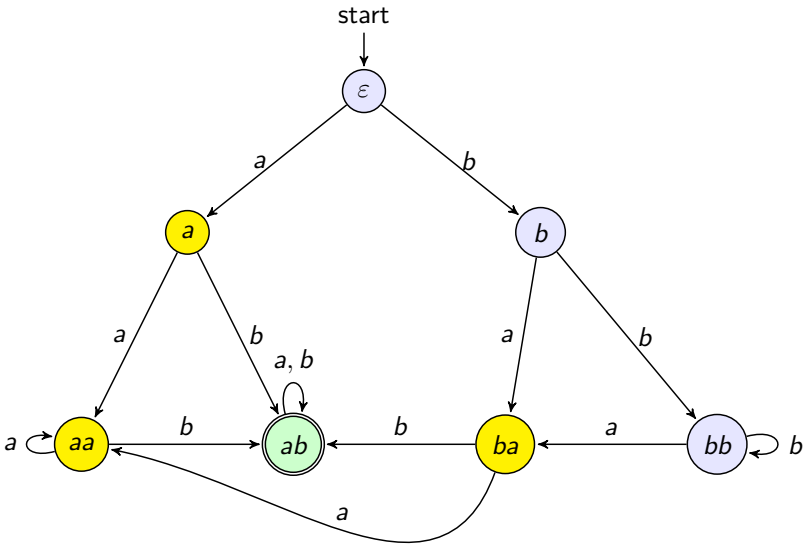
ab is the only accepting state, so every pair involving it is immediately distinguishable.



	ε	a	b	aa	ab	ba
a						
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

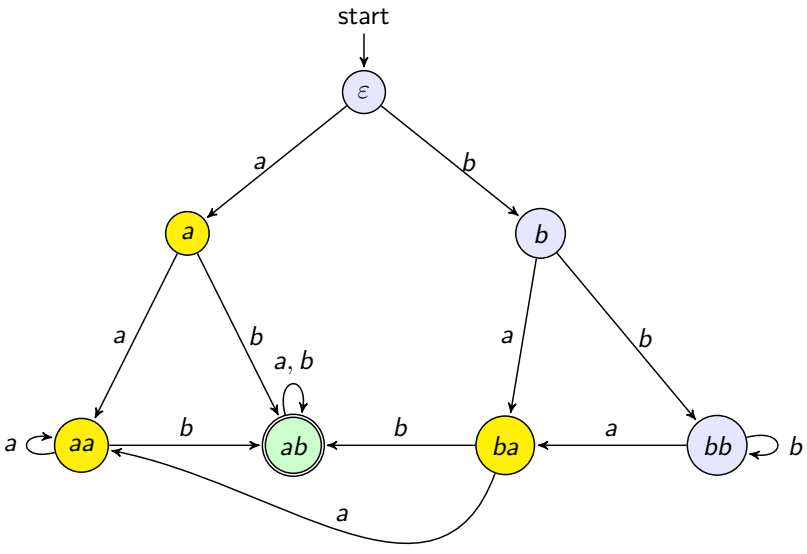
6 pairs marked. Everything else is open.

Round 1, row *a*



	ε	*a*	*b*	*aa*	*ab*	*ba*
a						
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

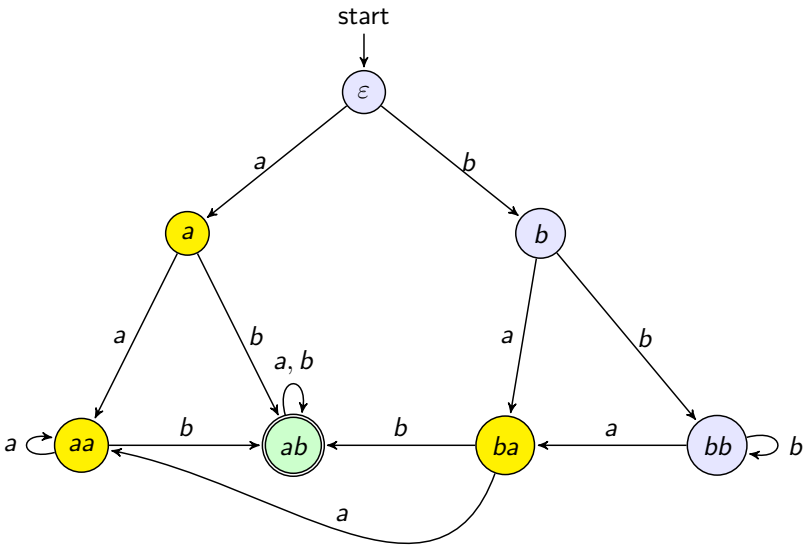
Round 1, row a



	ε	a	b	aa	ab	ba
a						
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

(a, ε): distinguished by b as (ab, b) marked.

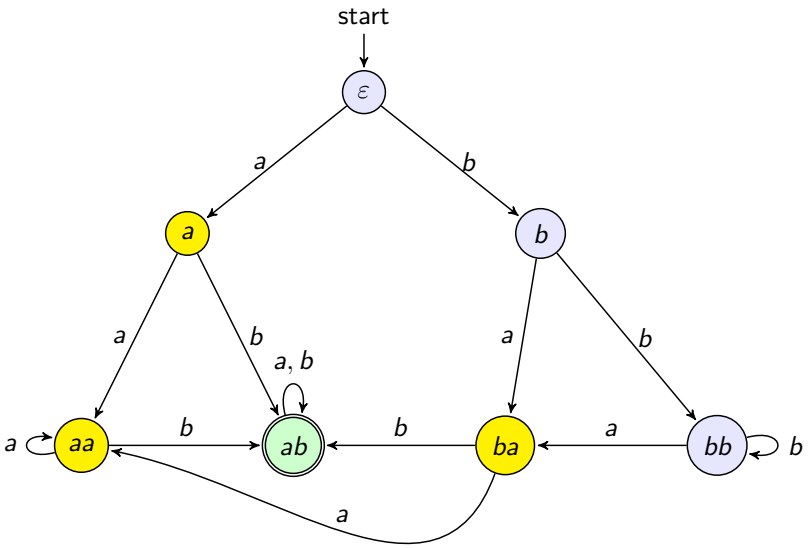
Round 1, row a



	ε	a	b	aa	ab	ba
a	X					
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

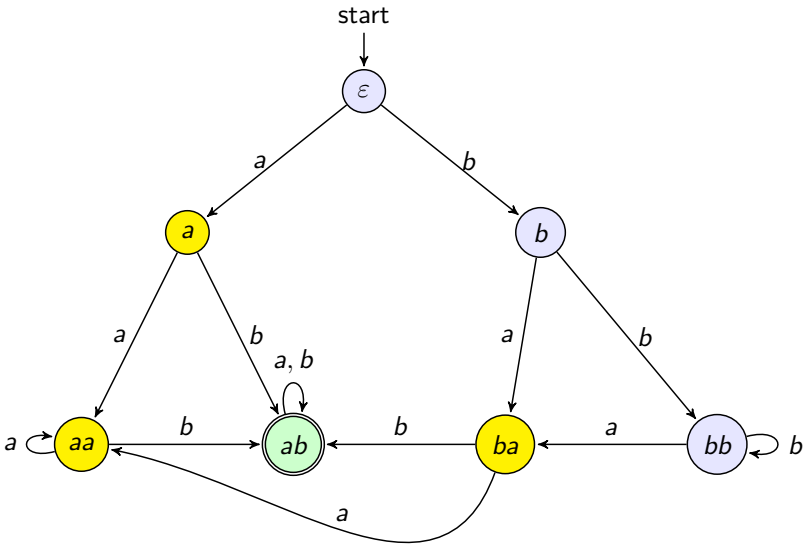
(a, ε): distinguished by b as (ab, b) marked.

Round 1, row b



	ε	a	b	aa	ab	ba
a	X					
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

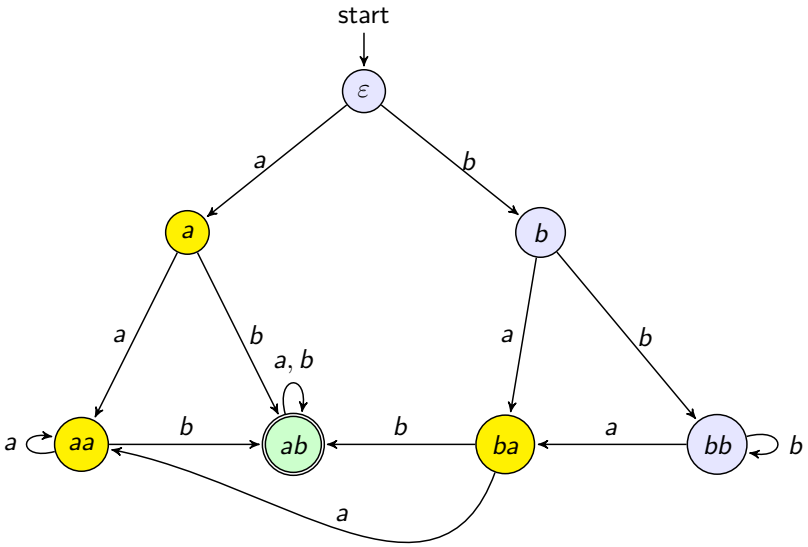
Round 1, row b



	ε	a	b	aa	ab	ba
a	X					
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

(b, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

Round 1, row b

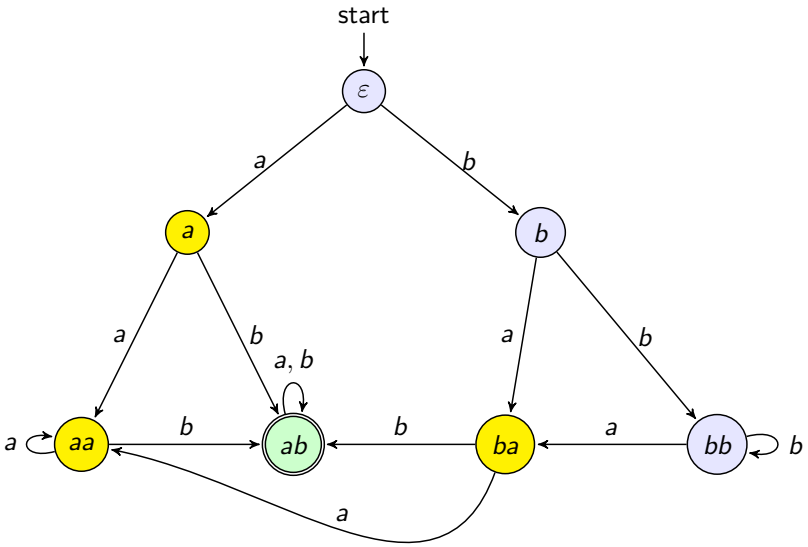


	ε	a	b	aa	ab	ba
a	X					
b						
aa						
ab	X	X	X	X		
ba					X	
bb					X	

(b, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(b, a): on a, (ba, aa) is unmarked. On b, (bb, ab) is marked. Marked.

Round 1, row b

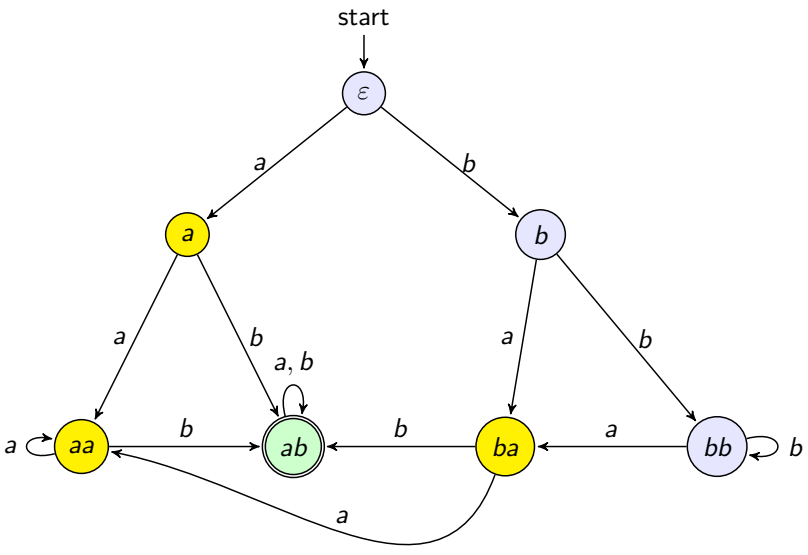


	ε	a	b	aa	ab	ba
a	X					
b		X				
aa						
ab	X	X	X	X		
ba					X	
bb					X	

(b, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

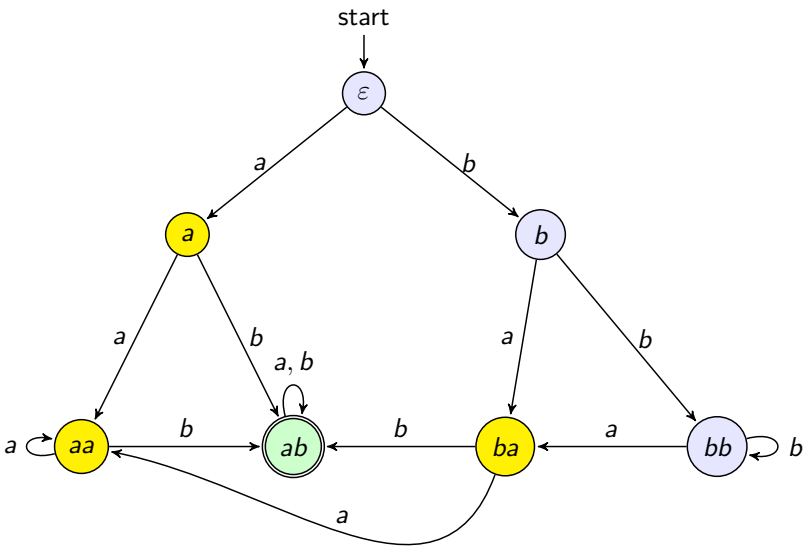
(b, a): on a, (ba, aa) is unmarked. On b, (bb, ab) is marked. Marked.

Round 1, row *aa*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa						
ab	X	X	X	X		
ba					X	
bb					X	

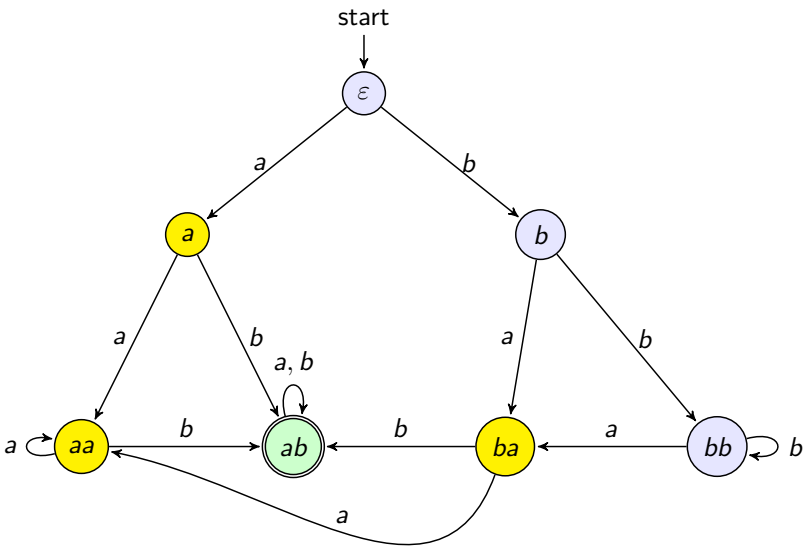
Round 1, row *aa*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa						
ab	X	X	X	X		
ba					X	
bb					X	

(aa, ε): on a, (aa, a) is unmarked. On b, (ab, b) is marked. Marked.

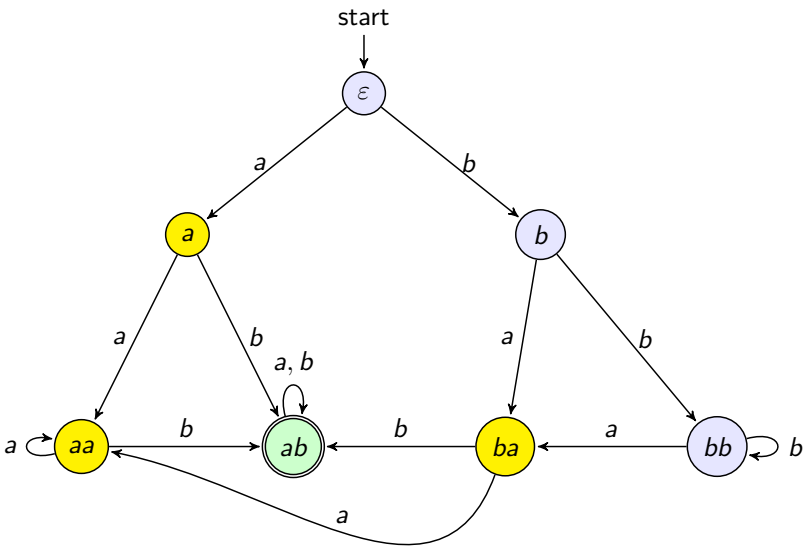
Round 1, row *aa*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X					
ab	X	X	X	X		
ba					X	
bb					X	

(aa, ε): on a, (aa, a) is unmarked. On b, (ab, b) is marked. Marked.

Round 1, row *aa*

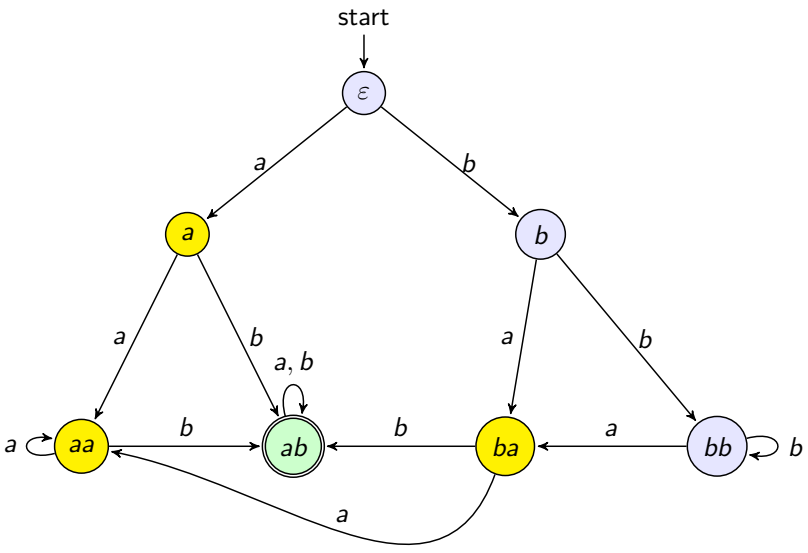


	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X					
ab	X	X	X	X		
ba					X	
bb					X	

(aa, ε): on a, (aa, a) is unmarked. On b, (ab, b) is marked. Marked.

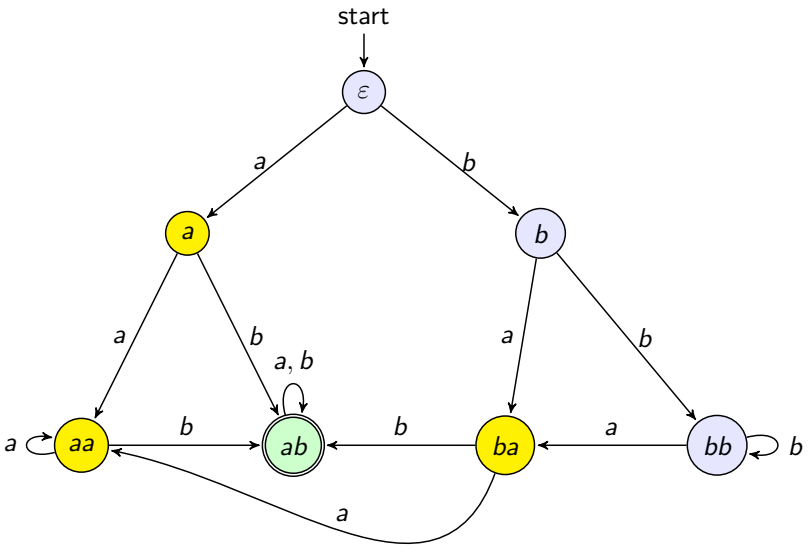
(aa, a): on a, we get (aa, aa). On b, we get (ab, ab). Stays unmarked.

Round 1, row *ba*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba					X	
bb					X	

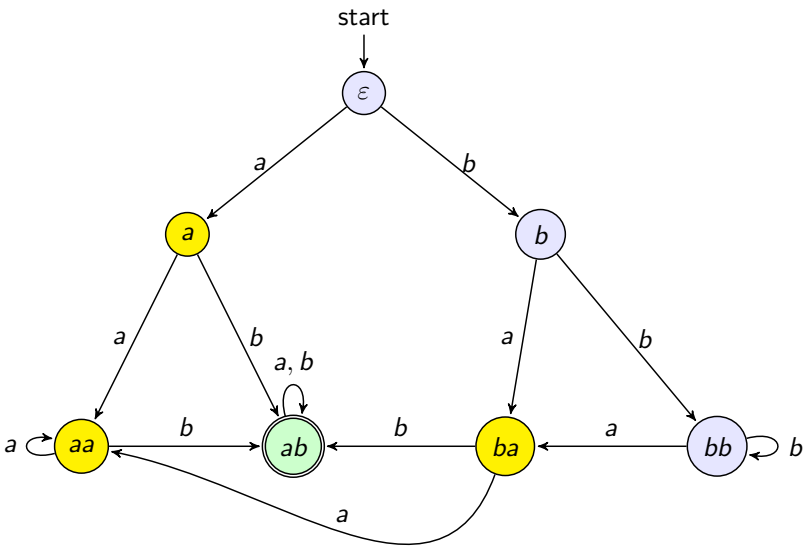
Round 1, row *ba*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba					X	
bb					X	

(ba, ε): on b, (ab, b) is marked. Marked.

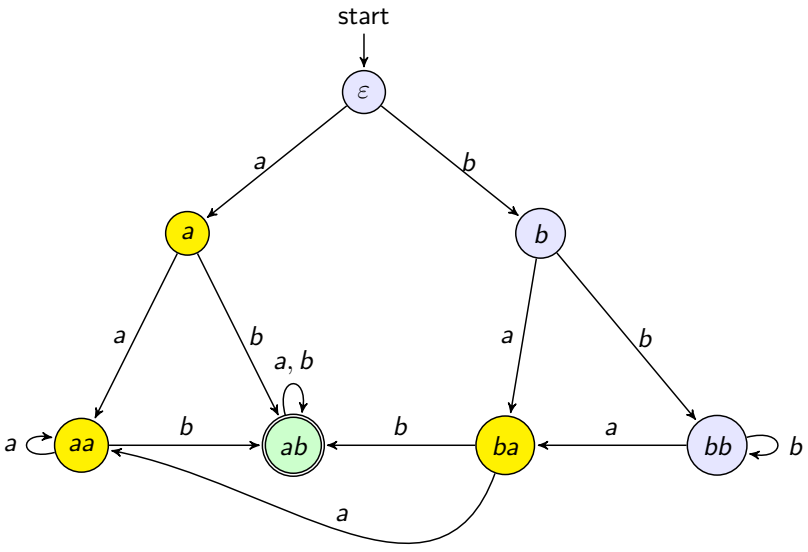
Round 1, row *ba*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X				X	
bb					X	

(ba, ε): on b, (ab, b) is marked. Marked.

Round 1, row *ba*

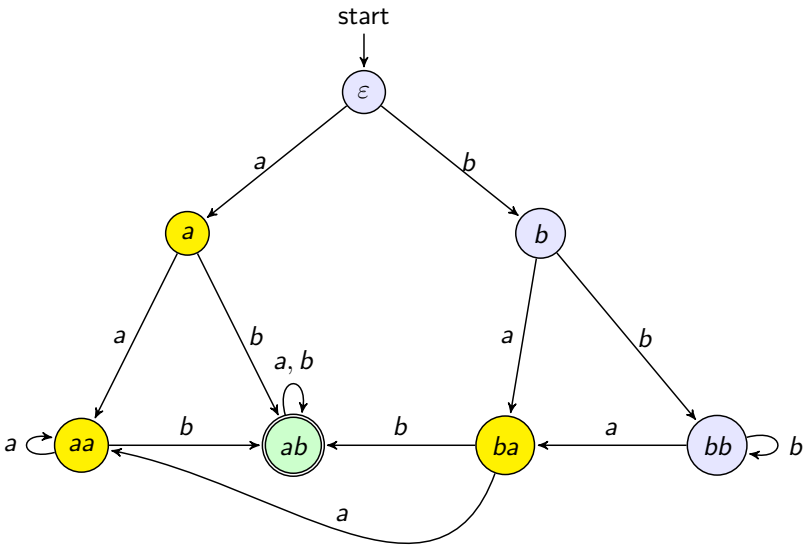


	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X				X	
bb					X	

(ba, ε): on b, (ab, b) is marked. Marked.

(ba, a): on a, (aa, aa). On b, (ab, ab). Stays unmarked.

Round 1, row *ba*



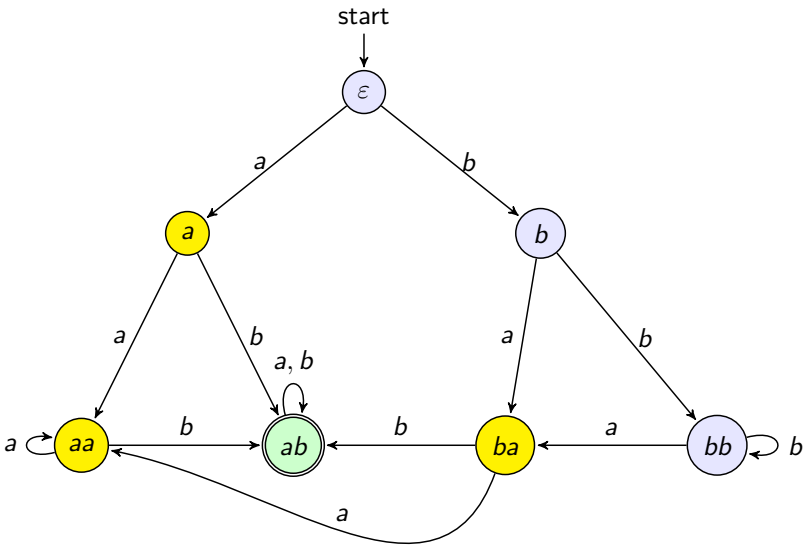
	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X				X	
bb					X	

(ba, ε): on b, (ab, b) is marked. Marked.

(ba, a): on a, (aa, aa). On b, (ab, ab). Stays unmarked.

(ba, b): on b, (bb, ab) is marked. Marked.

Round 1, row *ba*



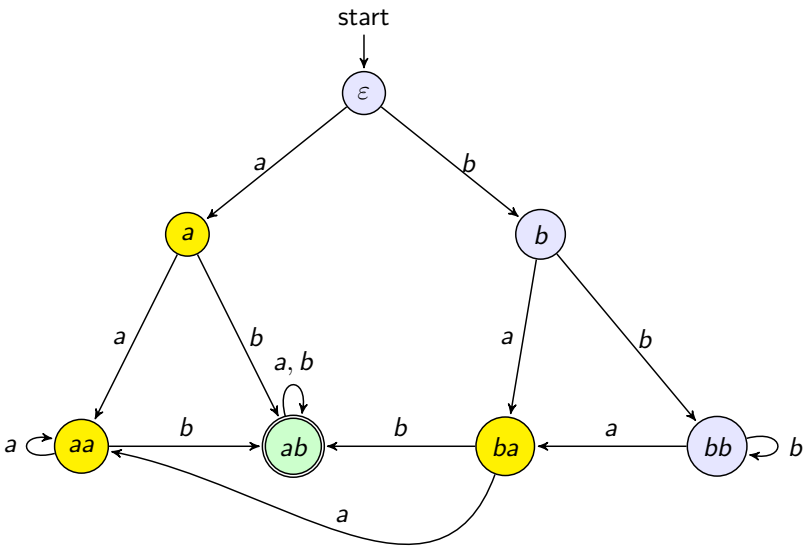
	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb					X	

(ba, ε): on b, (ab, b) is marked. Marked.

(ba, a): on a, (aa, aa). On b, (ab, ab). Stays unmarked.

(ba, b): on b, (bb, ab) is marked. Marked.

Round 1, row *ba*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb					X	

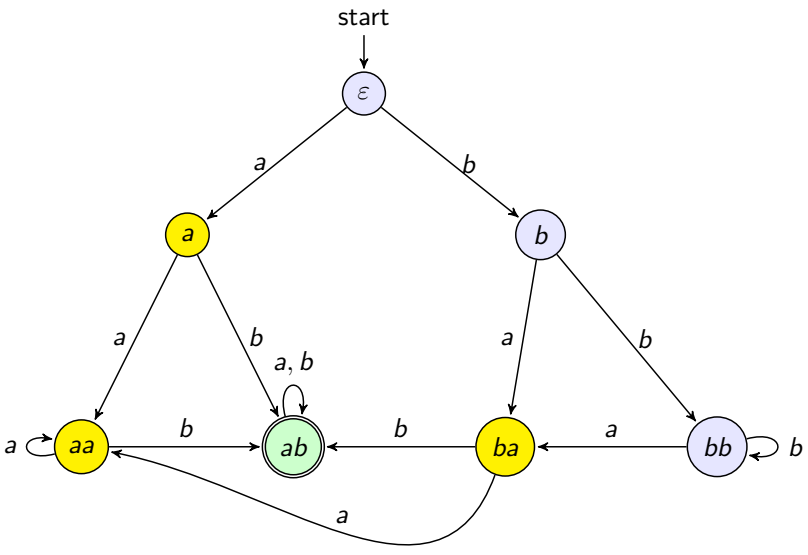
(ba, ε): on b, (ab, b) is marked. Marked.

(ba, a): on a, (aa, aa). On b, (ab, ab). Stays unmarked.

(ba, b): on b, (bb, ab) is marked. Marked.

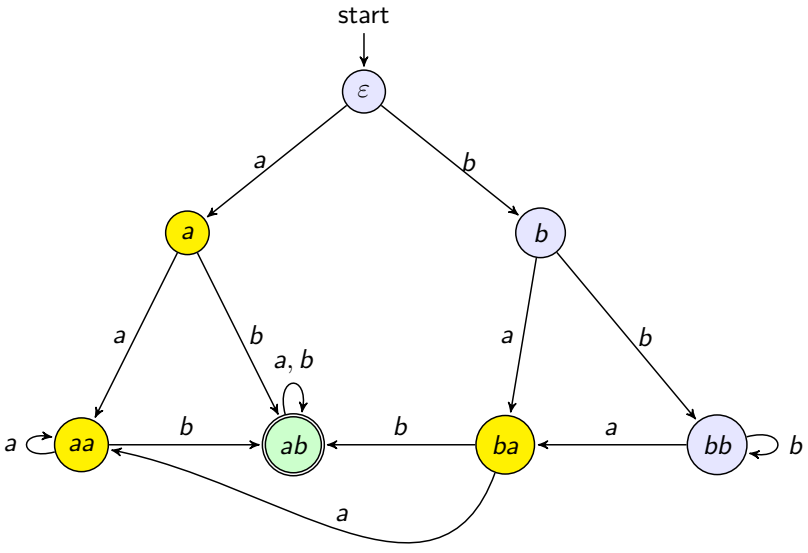
(ba, aa): on a, (aa, aa). On b, (ab, ab). Stays unmarked.

Round 1, row *bb*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb					X	

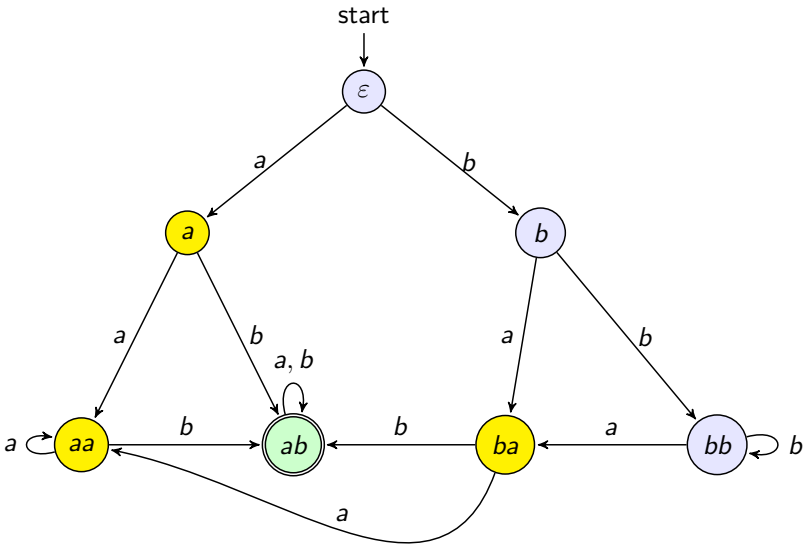
Round 1, row *bb*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb					X	

(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

Round 1, row *bb*

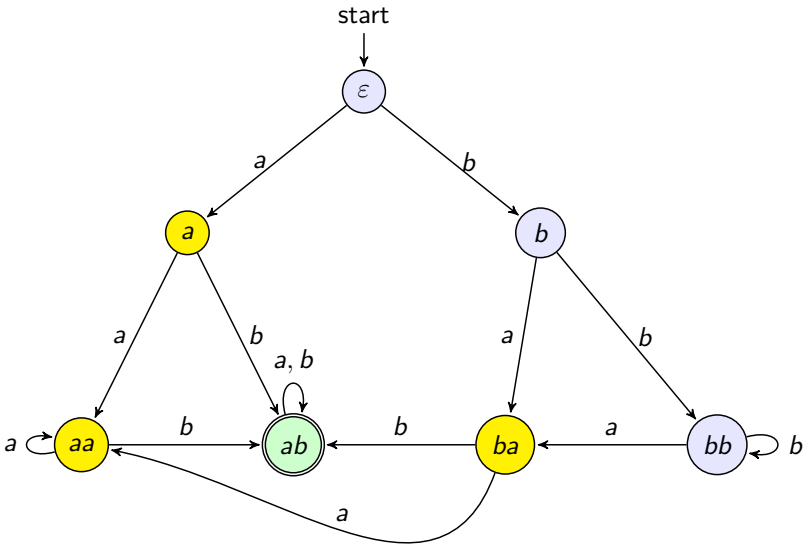


	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb					X	

(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(bb, a): on b, (bb, ab) is marked. Marked.

Round 1, row *bb*

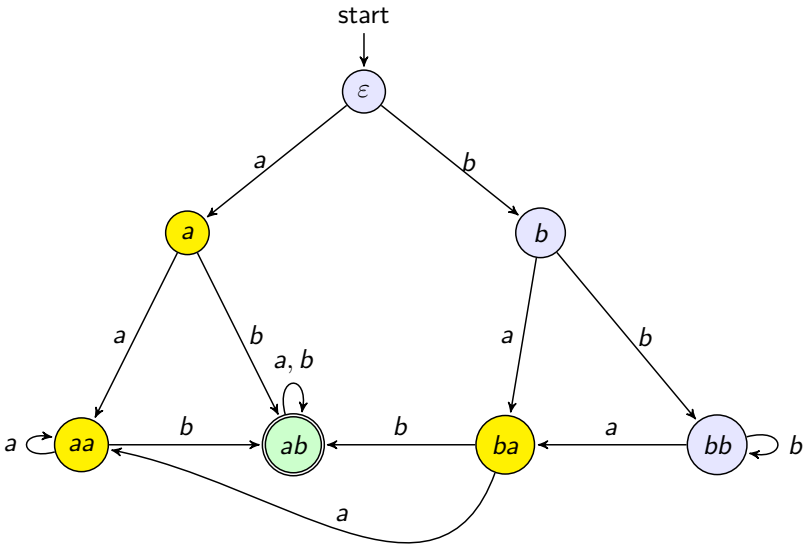


	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X			X	

(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(bb, a): on b, (bb, ab) is marked. Marked.

Round 1, row *bb*



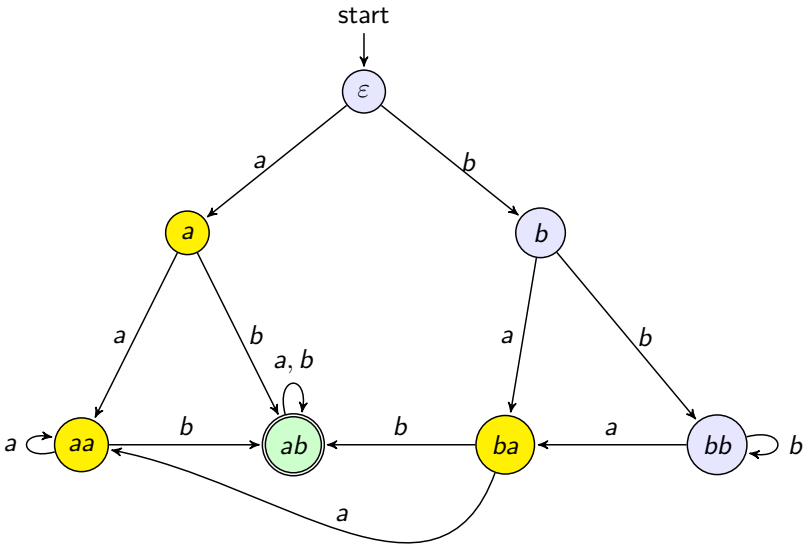
	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X			X	

(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(bb, a): on b, (bb, ab) is marked. Marked.

(bb, b): both transitions land on identical states. Stays unmarked.

Round 1, row *bb*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X			X	

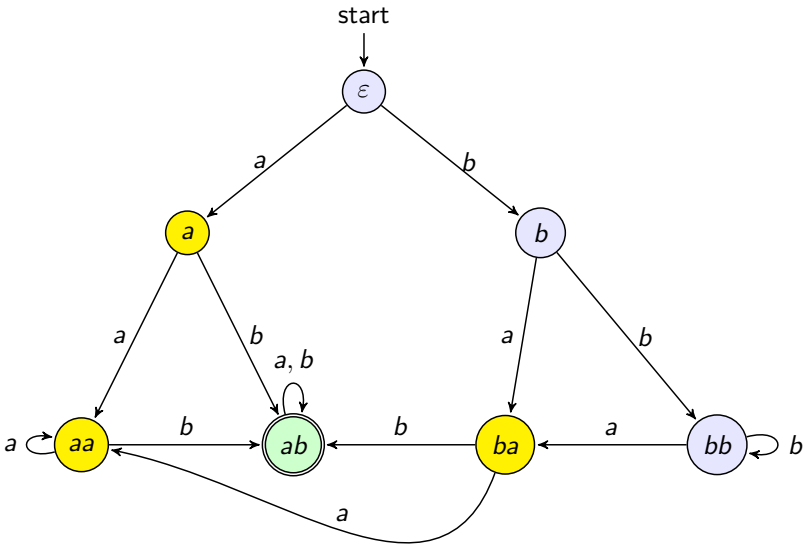
(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(bb, a): on b, (bb, ab) is marked. Marked.

(bb, b): both transitions land on identical states. Stays unmarked.

(bb, aa): on b, (bb, ab) is marked. Marked.

Round 1, row *bb*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X		X	X	

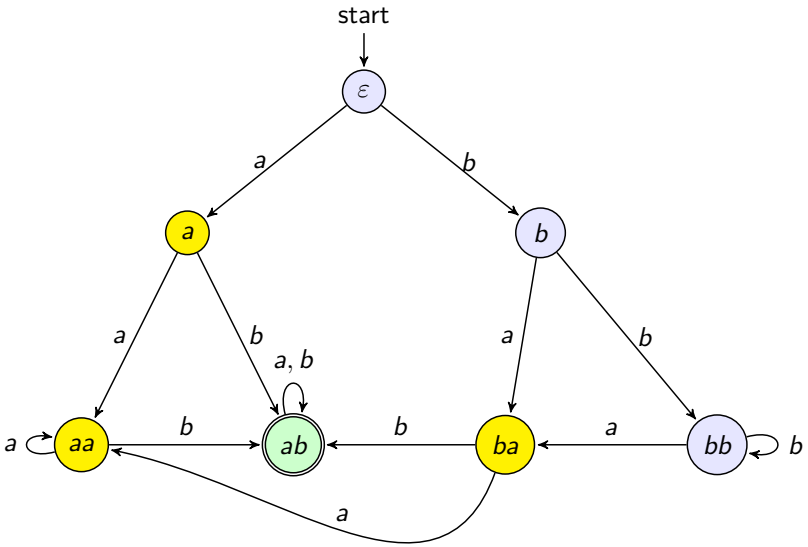
(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(bb, a): on b, (bb, ab) is marked. Marked.

(bb, b): both transitions land on identical states. Stays unmarked.

(bb, aa): on b, (bb, ab) is marked. Marked.

Round 1, row *bb*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X		X	X	

(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

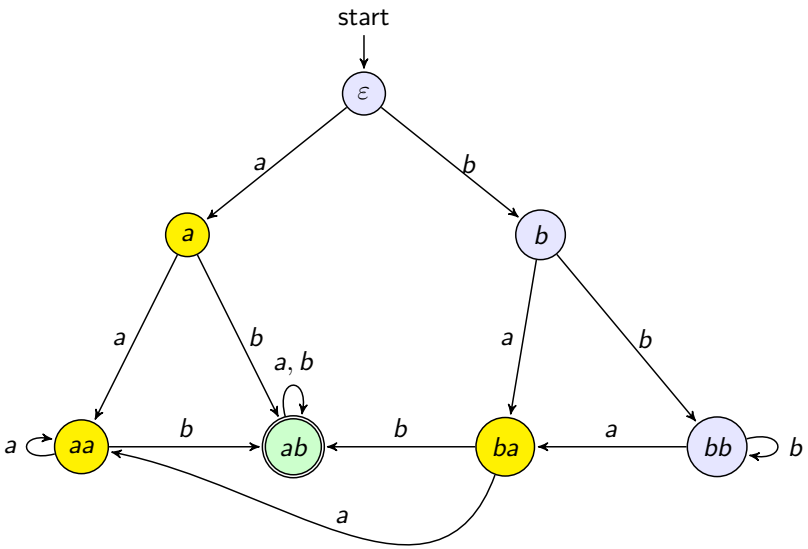
(bb, a): on b, (bb, ab) is marked. Marked.

(bb, b): both transitions land on identical states. Stays unmarked.

(bb, aa): on b, (bb, ab) is marked. Marked.

(bb, ba): on b, (bb, ab) is marked. Marked.

Round 1, row *bb*



	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X		X	X	X

(bb, ε): on a, (ba, a) is unmarked. On b, (bb, b) is unmarked. Stays unmarked.

(bb, a): on b, (bb, ab) is marked. Marked.

(bb, b): both transitions land on identical states. Stays unmarked.

(bb, aa): on b, (bb, ab) is marked. Marked.

(bb, ba): on b, (bb, ab) is marked. Marked.

Stable Table

One more pass finds nothing new

	ε	a	b	aa	ab	ba
a	X					
b		X				
aa	X		X			
ab	X	X	X	X		
ba	X		X		X	
bb		X		X	X	X

Stable Table

One more pass finds nothing new

	ε	a	b	aa	ab	ba
a	X					
b	=	X				
aa	X	=	X			
ab	X	X	X	X		
ba	X	=	X	=	X	
bb	=	X	=	X	X	X

$=$ marks the pairs that never got distinguished: $\{\varepsilon, b, bb\}$ and $\{a, aa, ba\}$.

Stable Table

One more pass finds nothing new

	ε	a	b	aa	ab	ba
a	X					
b	=	X				
aa	X	=	X			
ab	X	X	X	X		
ba	X	=	X	=	X	
bb	=	X	=	X	X	X

$=$ marks the pairs that never got distinguished: $\{\varepsilon, b, bb\}$ and $\{a, aa, ba\}$.

$\{\varepsilon, b, bb\}$, $\{a, aa, ba\}$ and $\{ab\}$ remain indistinguishable

Table-filling Algorithm

1. Distinguishable $= \{(p, q) \mid \text{exactly one of } p, q \in F\}$
2. Repeat until no new pair is added:
   - for every $a \in \Sigma$ and every pair (p, q):
     add (p, q) to Distinguishable if $(\delta(p, a), \delta(q, a)) \in$ Distinguishable.
3. Return Distinguishable.

Correctness

1. If two states are distinguished by the table-filling algorithm then they are not equivalent (obvious).
2. If two states are not distinguished by the table-filling algorithm then they are equivalent.

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- That is, (p, q) is distinguishable, but the algorithm did not find it
- Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q).

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- That is, (p, q) is distinguishable, but the algorithm did not find it
- Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q).
- Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q).

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- That is, (p, q) is distinguishable, but the algorithm did not find it
- Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q).
- Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q).
  - $w \neq \epsilon$ (why?)

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- That is, (p, q) is distinguishable, but the algorithm did not find it
- Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q).
- Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q).
  - $w \neq \epsilon$ (why?)
  - Let $w = ax$. As p, q are distinguishable, exactly one of $\hat{\delta}(p, ax), \hat{\delta}(q, ax)$ is accepting.
  - Then $p' = \delta(p, a)$ and $q' = \delta(q, a)$ are distinguished by x.

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- That is, (p, q) is distinguishable, but the algorithm did not find it
- Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q).
- Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q).
  - $w \neq \epsilon$ (why?)
  - Let $w = ax$. As p, q are distinguishable, exactly one of $\hat{\delta}(p, ax), \hat{\delta}(q, ax)$ is accepting.
  - Then $p' = \delta(p, a)$ and $q' = \delta(q, a)$ are distinguished by x.
  - If (p', q') was discovered by the algorithm, then (p, q) would have been discovered as well.

Correctness

- Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- That is, (p, q) is distinguishable, but the algorithm did not find it
- Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q).
- Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q).
  - $w \neq \epsilon$ (why?)
  - Let $w = ax$. As p, q are distinguishable, exactly one of $\hat{\delta}(p, ax), \hat{\delta}(q, ax)$ is accepting.
  - Then $p' = \delta(p, a)$ and $q' = \delta(q, a)$ are distinguished by x.
  - If (p', q') was discovered by the algorithm, then (p, q) would have been discovered as well.
  - If (p', q') is not discovered by the algorithm, then (p', q') is a bad pair with a shorter distinguishing word, a contradiction.

Minimization of DFAs

Let A be a DFA with no unreachable states

Let $\approx \subseteq Q \times Q$ be the state equivalence relation (computed say by the table-filling algorithm)

$$p \approx q \Leftrightarrow \forall x \in \Sigma^*(\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

- $\approx$ is an equivalence relation with finitely many classes
- Let $[q] = \{q' \mid q' \approx q\}$

Minimization of DFAs

Given DFA $A = (Q, \Sigma, \delta, q_0, F)$, we can minimize A to the DFA $A_{min} = (Q', \Sigma, \delta', q_0', F')$ called the *Quotient Automata* where

- $Q' = \{[q] \mid q \in Q\}$
- $\delta'([q], a) = [\delta(q, a)]$ for all $a \in \Sigma$
  If $p \approx q$ then $\delta(p, a) \approx \delta(q, a)$. $[p] = [q] \Rightarrow [\delta(p, a) = [\delta(q, a)]$
- $q_0' = [q_0]$
- $F' = \{[q] \mid q \in F\}$

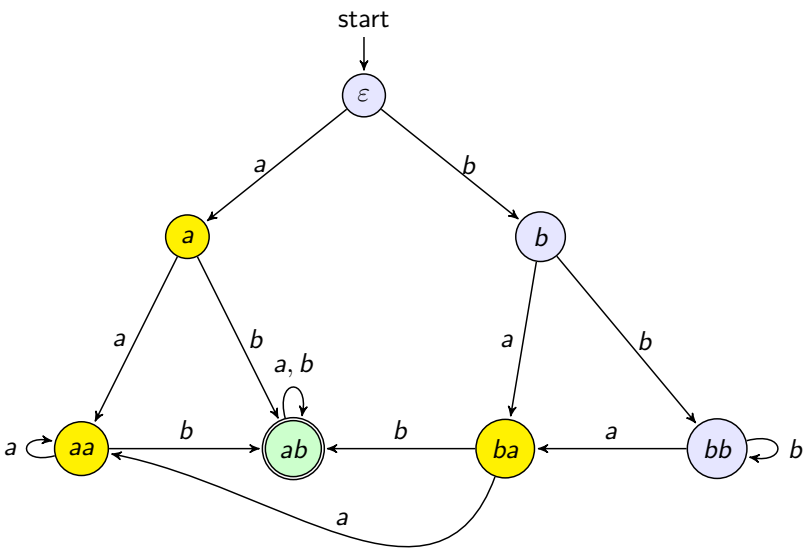
Minimization of DFAs

Given DFA $A = (Q, \Sigma, \delta, q_0, F)$, we can minimize A to the DFA $A_{min} = (Q', \Sigma, \delta', q_0', F')$ called the *Quotient Automata* where

- $Q' = \{[q] \mid q \in Q\}$
- $\delta'([q], a) = [\delta(q, a)]$ for all $a \in \Sigma$
  If $p \approx q$ then $\delta(p, a) \approx \delta(q, a)$. $[p] = [q] \Rightarrow [\delta(p, a) = [\delta(q, a)]$
- $q_0' = [q_0]$
- $F' = \{[q] \mid q \in F\}$

$q \in F$ iff $[q] \in F'$. If $p \approx q$ and $q \in F$, then $p \in F$. Each class is either inside F or disjoint from F.

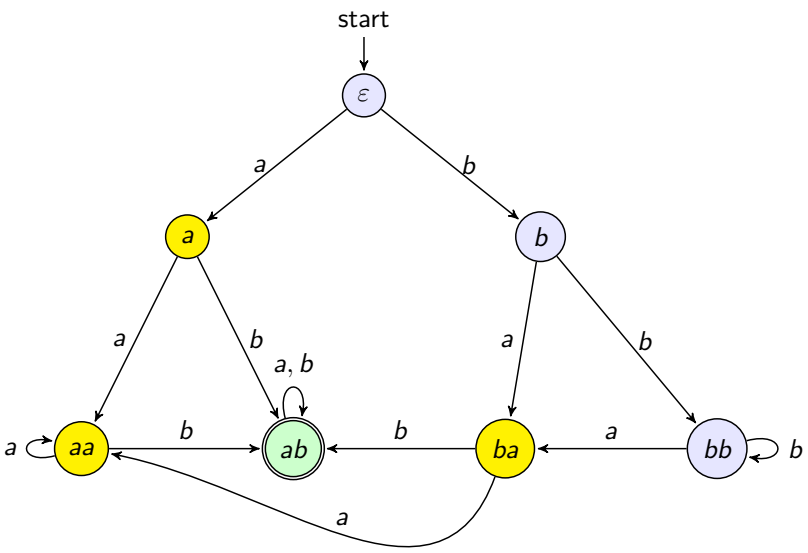
Quotient construction on our 7-state DFA



Classes from the table-filling algorithm:

$$C_0 = \{\varepsilon, b, bb\}, C_1 = \{a, aa, ba\}, C_2 = \{ab\}$$

Quotient construction on our 7-state DFA

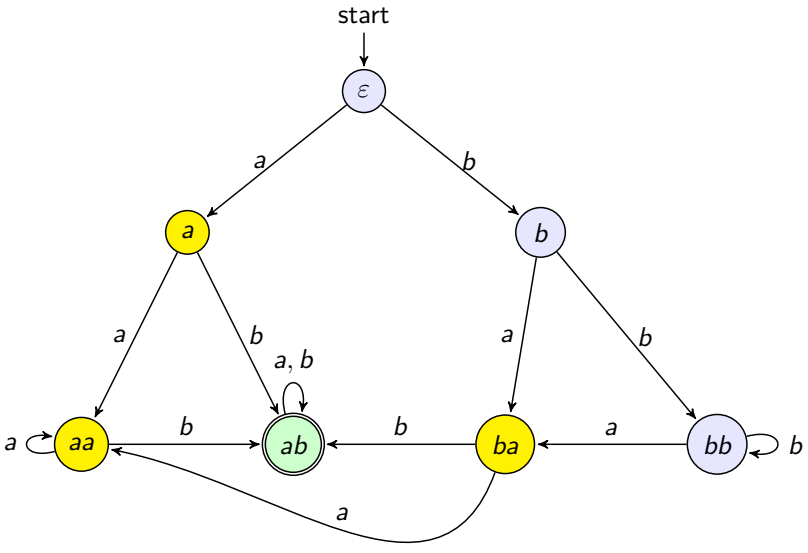


Classes from the table-filling algorithm:

$$C_0 = \{\varepsilon, b, bb\}, C_1 = \{a, aa, ba\}, C_2 = \{ab\}$$

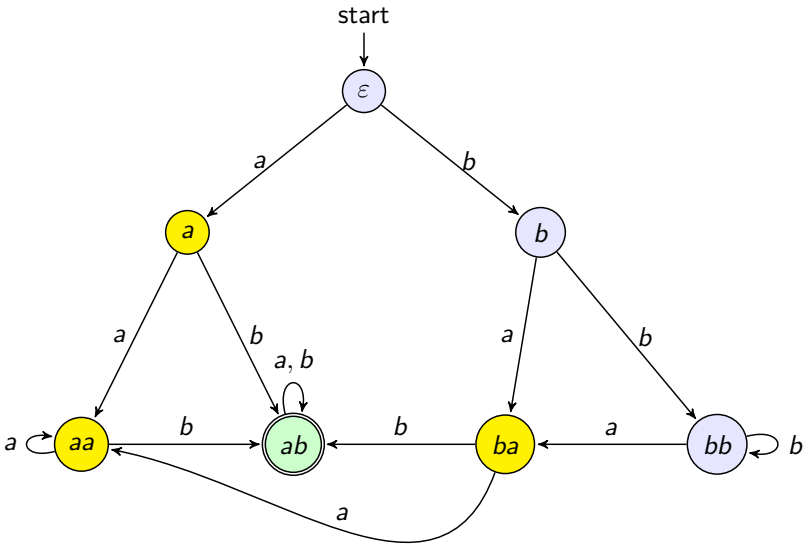
Is $\delta'([q], a) = [\delta(q, a)]$ well defined? does every state in a class send a (resp. b) into the *same* class, regardless of which representative we pick?

Quotient construction: class C_0



C_0 on a:
$\delta(\varepsilon, a)=a,\ \delta(b, a)=ba,\ \delta(bb, a)=ba$ all in C_1.

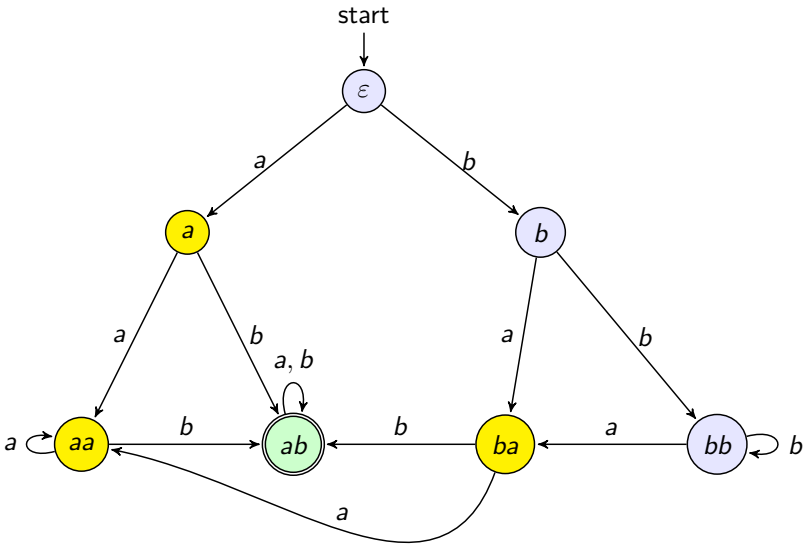
Quotient construction: class C_0



C_0 on a:
$\delta(\varepsilon, a)=a,\ \delta(b, a)=ba,\ \delta(bb, a)=ba$ all in C_1.

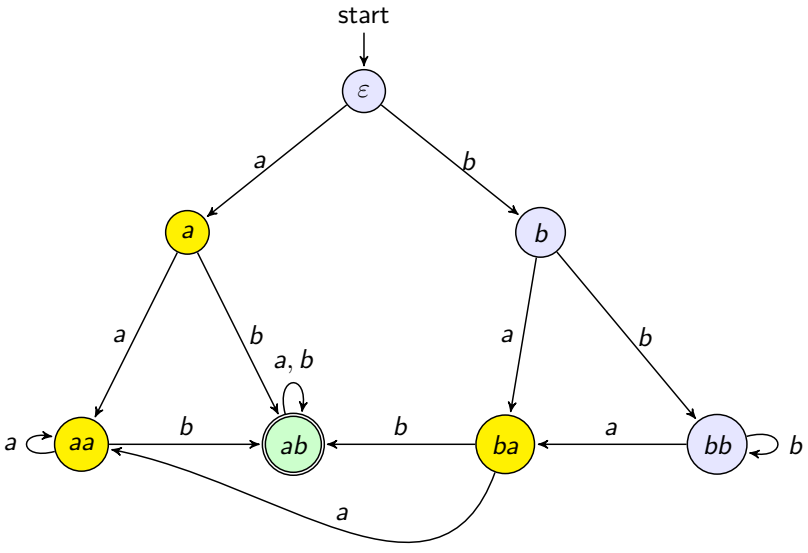
C_0 on b:
$\delta(\varepsilon, b)=b,\ \delta(b, b)=bb,\ \delta(bb, b)=bb$ all in C_0.

Quotient construction: class C_1



C_1 on a:
$\delta(a, a)=aa$, $\delta(aa, a)=aa$, $\delta(ba, a)=aa$ all in C_1.

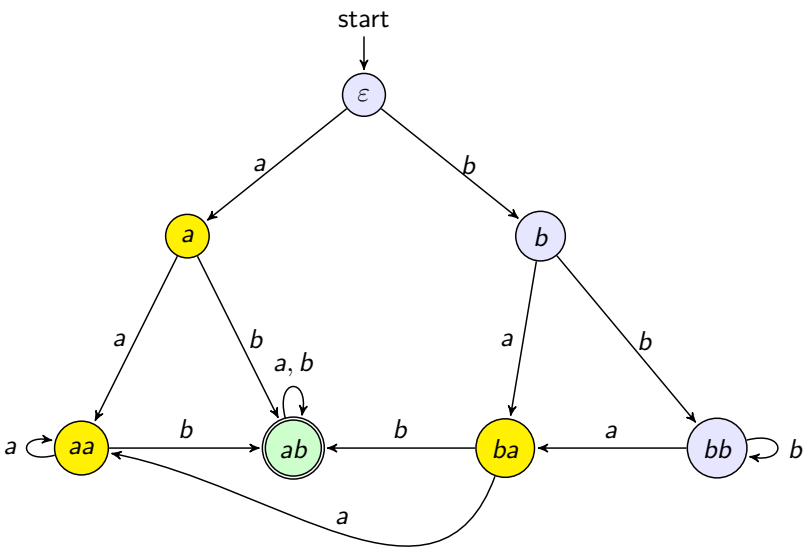
Quotient construction: class C_1



C_1 on a:
$\delta(a, a)=aa,\ \delta(aa, a)=aa,\ \delta(ba, a)=aa$ all in C_1.

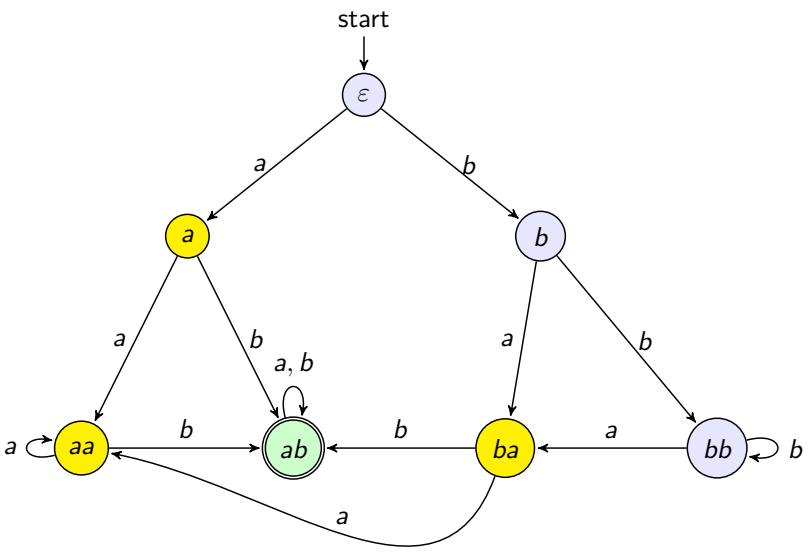
C_1 on b:
$\delta(a, b)=ab,\ \delta(aa, b)=ab,\ \delta(ba, b)=ab$ all in C_2.

Quotient construction: class C_2



C_2 on a, b: $\delta(ab, a)=\delta(ab, b)=ab$ stays in C_2.

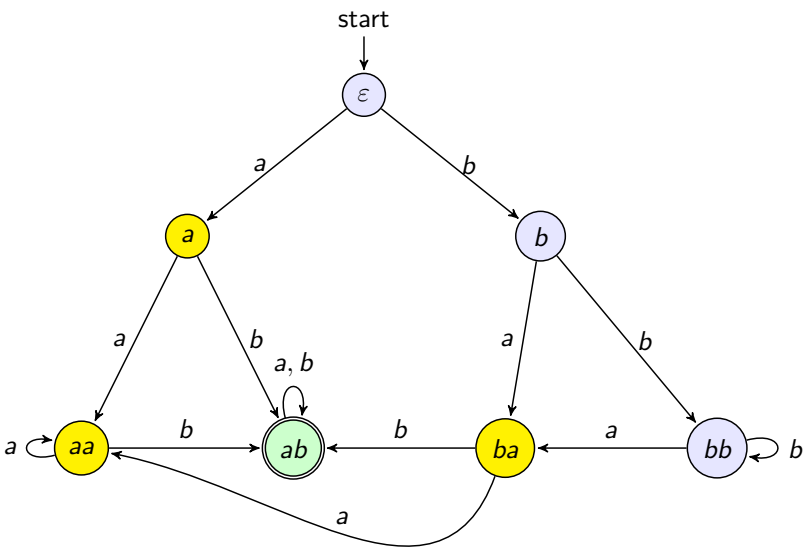
Quotient construction: class C_2

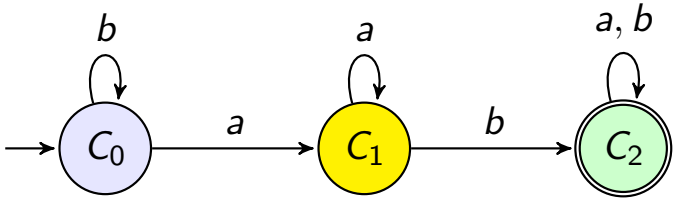


C_2 on a, b: $\delta(ab, a)=\delta(ab, b)=ab$ stays in C_2.

Every representative of a class agrees on which class it lands in : δ' is well defined.

Quotient construction: class C_2





C_2 on a, b: $\delta(ab, a)=\delta(ab, b)=ab$ stays in C_2.

Every representative of a class agrees on which class it lands in : δ' is well defined.

So the quotient automaton A_{min} is exactly this 3-state machine, with states C_0, C_1, C_2.

Minimization of DFAs

$L(A_{min}) = L(A)$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q_0', x) \in F'$ iff

Minimization of DFAs

$L(A_{min}) = L(A)$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q_0', x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff

Minimization of DFAs

$L(A_{min}) = L(A)$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q_0', x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff

Minimization of DFAs

$L(A_{min}) = L(A)$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q_0', x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff
4. $\hat{\delta}(q_0, x) \in F$ iff

Minimization of DFAs

$L(A_{min}) = L(A)$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q_0', x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff
4. $\hat{\delta}(q_0, x) \in F$ iff
5. $x \in L(A)$

Minimization of DFAs

$L(A_{min}) = L(A)$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q_0', x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff
4. $\hat{\delta}(q_0, x) \in F$ iff
5. $x \in L(A)$

Claim

1. No two distinct states in A_{min} are equivalent.
2. A_{min} is the minimum and unique (upto state renaming) DFA equivalent to A.

Minimality of A_{min}

- Assume there is a DFA B with smaller number of states than A_{min}, such that $L(B) = L(A)$.

Minimality of A_{min}

- Assume there is a DFA B with smaller number of states than A_{min}, such that $L(B) = L(A)$.
- Using the table-filling algorithm, compute equivalent states of B and A_{min}.

Minimality of A_{min}

- Assume there is a DFA B with smaller number of states than A_{min}, such that $L(B) = L(A)$.
- Using the table-filling algorithm, compute equivalent states of B and A_{min}.
- The initial states of B and A_{min} must be equivalent (why?)

Minimality of A_{min}

- Assume there is a DFA B with smaller number of states than A_{min}, such that $L(B) = L(A)$.
- Using the table-filling algorithm, compute equivalent states of B and A_{min}.
- The initial states of B and A_{min} must be equivalent (why?)
- After reading any $w \in \Sigma^*$ from their initial states, both DFAs enter equivalent states (why?)

Minimality of A_{min}

- Assume there is a DFA B with smaller number of states than A_{min}, such that $L(B) = L(A)$.
- Using the table-filling algorithm, compute equivalent states of B and A_{min}.
- The initial states of B and A_{min} must be equivalent (why?)
- After reading any $w \in \Sigma^*$ from their initial states, both DFAs enter equivalent states (why?)
- For every state of A_{min}, there is an equivalent state in B
- Since the number of states in B are $<$ than those in A_{min}, there are at least two states p, q in A_{min} equivalent to some state in B.
- That is, p, q are equivalent, a contradiction.